```
def update(self, frame, rate = 0.125):
    (x, y), (w, h) = self.pos, self.size
    self.last_img = img = cv.getRectSubPix(frame, (w, h), (x, y))
    img = self.preprocess(img)
    self.last_resp, (dx, dy), self.psr = self.correlate(img)
    self.good = self.psr > 8.0
    if not self.good:
        return



    self.last_img = img = cv.getRectSubPix(frame, (w, h), self.pos)
    img = self.preprocess(img)
    A = cv.dft(img, flags=cv.DFT_COMPLEX_OUTPUT)
    H1 = cv.mulSpectrums(self.G, A, 0, conjB=True)
    H2 = cv.mulSpectrums(     A, A, 0, conjB=True)
    self.H1 = self.H1 * (1.0-rate) + H1 * rate
    self.H2 = self.H2 * (1.0-rate) + H2 * rate
    self.update_kernel()

@property
def state_vis(self):
    f = cv.idft(self.H, flags=cv.DFT_SCALE | cv.DFT_REAL_OUTPUT )
    h, w = f.shape
    f = np.roll(f, -h//2, 0)
    f = np.roll(f, -w//2, 1)
    kernel = np.uint8( (f-f.min()) / f.ptp()*255 )
    resp = self.last_resp
    resp = np.uint8(np.clip(resp/resp.max(), 0, 1)*255)
    vis = np.hstack([self.last_img, kernel, resp])
    return vis
```

```
def draw_state(self, vis):
    (x, y), (w, h) = self.pos, self.size
    x1, y1, x2, y2 = int(x-0.5*w), int(y-0.5*h), int(x+0.5*w), int(y+0.5*h)
    cv.rectangle(vis, (x1, y1), (x2, y2), (0, 0, 255))
    if self.good:
        cv.circle(vis, (int(x), int(y)), 2, (0, 0, 255), -1)
        #print(x,y)
        self.pos = x, y
        print(self.pos )
        #mouse.move(x+dx,y+dy)
        xmove=mapfnx(x)
        ymove= mapfny(y)
        print (xmove)
        print(ymove)
        mouse.position = (xmove,ymove)


        #mouse.position = (x,y)
```

Fig. 9: Code for controlling mouse using converted coordinates data

```
cv.imshow('frame', vis)
ch = cv.waitKey(10)
if ch == 27:
    break
if ch == ord(' '):
    self.paused = not self.paused
if ch == ord('c'):
    self.trackers = []
if button.is_pressed:
    press=0
   # print("eyeblink")
    for x in range(0, 5):
        if button.is_pressed:
            press =press+1
            #print(press)
            if press==5:
                mouse.click(Button.left, 2)

                #print("mousepress")
```

Fig. 10: Eye blink converted into mouse click on GUI

Detecting eye movement using image processing. For a mouse cursor to move, the human eye needs to refer to an object and move the mouse relative to it. Using eye movement, a physically challenged person can effortlessly operate a PC. But image processing of eye movement does not give accurate results, does not work in low light or completely dark conditions, and the entire process is quite difficult.

Thus, a light mounted on the sensor can be used to track eye movement whenever the person moves his/her head.

Translating eye movement and blink to control the PC. This solution detects eye blinks and moves the light mounted on the eyeglass. But it is useless without recording this movement on the PC's GUI to obtain accurate movement of the mouse cursor. While it needs to cover the entire length and width of a PC monitor, a human head can move only up to a certain degree. To solve this problem, a small head movement needs to be converted into large pixel movement for the mouse cursor.

Prerequisites

To develop the basics, prepare the SD card with the latest Raspbian OS and check whether it has a pre-installed Python IDLE. Next, install the following Python libraries and modules for the project:

- OpenCV
- pynput
- numpy
- gpiozero

To do so, open the terminal and use the following commands:

```
sudo pip3 install python-opencv
sudo pip3 install numpy
sudo pip3 install gpiozero
sudo pip3 install pynput
sudo pip3 install nmap
```

After installing all the libraries, include the OpenCV official GitHub repository inside the Raspberry Pi using the following command:

```
git clone https://github.com/opencv/
opencv
```

We are now ready to code.

Coding

You will need to modify the example code found in the OpenCV library folder and fuse your code in it to prepare the device. For that, open the OpenCV folder→platforms folder→python, and select the mouse.py code. Copy and paste it in a new text file named headcontrol-GUI.py and save it.

You will need to get the eye blink sequence from the sensor for a mouse click. To get it, access the gpio pins and import the gpiozero module into the code to read the